

# Mid-Term Assignment Report: Custom RAG Pipeline on an AI Course Textbook

Author: Kosta | Date: 2026-05-26

This report documents a complete Retrieval-Augmented Generation (RAG) pipeline built over a 152-page AI Developers Course Textbook (PDF). The system implements the full pipeline (loading → cleaning → chunking → embedding → indexing → retrieval → generation → citation → evaluation) and exposes the required `answer(question)` interface. Three retrieval strategies are compared: dense vector search, BM25 lexical retrieval, and a hybrid RRF fusion of both.

## 1. Corpus Description

The corpus is a single structured PDF: **AI Developers Course Textbook v2** (257 KB, 152 pages). It covers twelve modules of an AI/ML curriculum:

| Module | Topic                               |
|--------|-------------------------------------|
| 1      | Introduction to ML Paradigms        |
| 2      | Linear Models & Optimization        |
| 3      | Trees & Ensembles                   |
| 4      | Neural Networks & Classification    |
| 5      | NLP Basics & Preprocessing          |
| 6      | Word Embeddings & Vectors           |
| 7      | Transformer Architecture            |
| 8      | LLMs & Prompt Engineering           |
| 9      | Introduction to RAG                 |
| 10     | Data Ingestion & Chunking           |
| 11     | Vector Stores & Retrieval           |
| 12     | Project Implementation & Evaluation |

**Suitability for RAG.** The textbook is structured with explicit module/concept/subsection headings, enabling precise provenance tracking. A baseline LLM may know general ML concepts but lacks the exact module-level structure, code examples, and numerical details present in this specific document, making retrieval load-bearing for precise answers. Rich structural metadata (module, concept, subsection, page range) enables fine-grained citation and grounding.

**Extraction.** Text is extracted page-by-page via PyMuPDF (`fitz`). Font-size heuristics identify module markers ( $\geq 28$  pt), section headings (17–21 pt), named subsections (13.5–16 pt), and body text ( $\leq 11$  pt). Page-number lines are discarded. Total indexed chunks: **182** (structured mode).

## 2. System Architecture

End-to-end pipeline:

```
PDF → load_pdf_structured() → clean_text() → chunk_documents() → SentenceTransformer
    encode → Chroma (HNSW) → Retriever [dense | BM25 | hybrid-RRF] → build_prompt() → Ollama
    (gemma3:4b) → parse_citations() → answer()
```

Concrete components: three document loaders and two chunkers in `src/utils.py`; index builder in `src/build_index.py`; three retriever classes (`Retriever`, `BM25Retriever`, `HybridRetriever`) in `src/retrieval.py`; LLM generation and citation parsing in `src/generation.py`; and the assignment-required `answer()` entry point in `src/rag_system.py`. Evaluation tooling lives in `eval/build_gold_set.py` and `eval/run_eval.py`.

## 3. Chunking Strategy

Three document *load modes* control document granularity before chunking, selectable via `--load-mode`:

| Load Mode            | Granularity                                          | Chunks                |
|----------------------|------------------------------------------------------|-----------------------|
| full                 | One document for entire PDF                          | Depends on chunk size |
| page                 | One document per page                                | Up to 152 docs        |
| structured (default) | One doc per logical section via font-size heuristics | 182 sections          |

Two *chunking strategies* are then applied, selectable via `--chunk-strategy`:

| Strategy        | Parameters                       | Behavior                                                                                             |
|-----------------|----------------------------------|------------------------------------------------------------------------------------------------------|
| fixed (default) | chunk_size=500 words, overlap=80 | Word-level sliding window. Cheap and predictable.                                                    |
| sentence        | max_words=120, overlap_sents=1   | Greedy sentence packing up to a word budget with one-sentence overlap. Preserves natural boundaries. |

**Why this design.** In structured mode the textbook sections average ~80 words, so nearly every section maps to a single chunk regardless of strategy — chunking parameters are effectively inert at this granularity. The sentence strategy is available for full-PDF or page modes where multi-paragraph sections can exceed the word budget.

**Concrete example where chunking helped.** Module 9 (Introduction to RAG) spans multiple subsections including Theoretical Foundation and Python Implementation. Structured load keeps each subsection as a separate chunk with dedicated metadata, so a question about the implementation details retrieves the correct subsection rather than being drowned out by the broader theoretical content.

**Where chunking is limited.** Very short subsections (under 30 words) produce embeddings dominated by structural boilerplate ("Knowledge Check", "Learning Objective"). These chunks may surface on broad queries even when content-irrelevant, adding noise to top-k results.

## 4. Embedding & Vector Index

Embeddings: sentence-transformers/all-MiniLM-L6-v2 (384-d, L2-normalized). Chosen for its strong cost/quality tradeoff on a CPU-only setup — encoding 182 chunks completes in seconds on a laptop CPU with no GPU required. The model is LRU-cached across retrieval calls.

Vector store: Chroma with HNSW and cosine distance, persisted under `data/processed/chroma/`. Each document stores the embedding, chunk text, and a metadata dict (`doc_id`, `chunk_id`, `source`, `page_start`, `page_end`, `module`, `concept`, `subsection`). Storing structural metadata at index time allows the LLM to produce grounded citations with module-level provenance without re-fetching the source document.

**Reproducibility.** Deleting `data/processed/chroma` and rerunning `python src/build_index.py --clean` rebuilds an identical index. Chunk IDs are deterministic strings of the form `pdf_<filename>_sec<N>_chunk_<i>`.

## 5. Retrieval

Three retrieval strategies are implemented, all returning `{chunk_id, text, score, metadata}` dicts. Default top-k = 5, configurable via `RAG_TOP_K` or the `k` argument to `answer()`.

| Class           | Method                                       | Description                                                                                                           |
|-----------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Retriever       | Dense (Chroma + MiniLM)                      | Encodes query with MiniLM, queries Chroma HNSW index. <code>score = 1 - cosine_distance</code> .                      |
| BM25Retriever   | Sparse ( <code>rank_bm25</code> , in-memory) | Tokenises on <code>[A-Za-z0-9_]+</code> , lowercased. Filters zero-score results. Build time <50 ms.                  |
| HybridRetriever | RRF fusion of Dense + BM25                   | Reciprocal Rank Fusion ( $K=60$ ). Retrieves 4×k from each retriever, fuses, re-ranks. No score normalisation needed. |

**Reported metrics.** **Hit@k**: fraction of grounded questions where at least one retrieved chunk shares a `chunk_id` or `doc_id` with the gold entry. **MRR@k**: mean reciprocal rank of the first correct result. **Answer Accuracy**: token-level

F1 (stop-words removed) between generated answer and reference answer, sampled over the first 30 grounded questions. Negation/absence questions are evaluated separately (correct iff the system returns the refusal string).

## 6. Prompt Design

The system prompt enforces five hard rules: (1) answer only from the provided context chunks, (2) say exactly “The information was not found in the provided context.” if absent, (3) cite chunks inline using [chunk\_id] in square brackets, (4) never invent facts, numbers, or names, (5) keep answers concise (1–4 sentences unless a list is required).

Context is formatted as one block per chunk with a structured header:

```
[chunk_id] (section=module > concept > subsection, pages=p<start>-<end>)
```

This surfaces the structural hierarchy (module, concept, subsection) and page range so the LLM can reason about provenance without additional look-ups. Citations are extracted from the LLM reply with the regex

`\[ ([^\[\]\s, ]+_chunk_\d+)\]` and intersected with the set of retrieved IDs — any ID the model invents but did not receive in context is silently dropped.

## 7. Evaluation Results

Gold set: ~182 questions (one per chunk) generated by `eval/build_gold_set.py` using the same gemma3:4b LLM. Questions cycle through five required categories (factual, numerical, temporal, comparison, negation/absence) with a fixed seed for reproducibility. Negation entries carry empty `must_cite_chunk_ids` by design.

**Retrieval and generation metrics across all ablation configs:**

| Config              | Retriever  | k | Chunks | Hit@k | MRR@k | Ans Acc |
|---------------------|------------|---|--------|-------|-------|---------|
| dense_k3            | dense      | 3 | 182    | 0.620 | 0.522 | 0.261   |
| dense_k5 (baseline) | dense      | 5 | 182    | 0.672 | 0.532 | 0.287   |
| dense_k8            | dense      | 8 | 182    | 0.693 | 0.535 | 0.294   |
| bm25_k5             | BM25       | 5 | 182    | 0.679 | 0.609 | 0.364   |
| hybrid_k5           | hybrid RRF | 5 | 182    | 0.693 | 0.588 | 0.340   |

**Manual answer classification** (10 sampled answers from `eval/answers_sample.jsonl`):

| Class             | Count | Notes                                                                                 |
|-------------------|-------|---------------------------------------------------------------------------------------|
| Correct           | 7     | Right answer with valid in-context citation.                                          |
| Partially correct | 0     |                                                                                       |
| Incorrect         | 2     | Generation returned refusal despite gold chunk appearing in top-5 (retrieval noise).  |
| Hallucinated      | 1     | Model cited a chunk ID not present in the retrieved set (dropped by citation filter). |

## 8. Ablation Study

Two ablation axes were varied: **retriever type** (dense, BM25, hybrid) and **top-k** (3, 5, 8 with dense retrieval). Chunk size was not varied because in structured load mode all sections average ~80 words — both 300-word and 700-word windows produce identical indices.

| Experiment            | Hit@k | MRR@k | Ans Acc | Key finding                                             |
|-----------------------|-------|-------|---------|---------------------------------------------------------|
| dense, k=3            | 0.620 | 0.522 | 0.261   | Focused window; misses when gold chunk ranked 4+.       |
| dense, k=5 (baseline) | 0.672 | 0.532 | 0.287   | Balanced precision/recall baseline.                     |
| dense, k=8            | 0.693 | 0.535 | 0.294   | Marginal recall gain; MRR barely moves.                 |
| BM25, k=5             | 0.679 | 0.609 | 0.364   | Best MRR and answer accuracy. Exact-term queries excel. |

|                 |       |       |       |                                                        |
|-----------------|-------|-------|-------|--------------------------------------------------------|
| Hybrid RRF, k=5 | 0.693 | 0.588 | 0.340 | Best Hit@k (tied dense_k8). RRF smooths BM25 sparsity. |
|-----------------|-------|-------|-------|--------------------------------------------------------|

**Reading the results.** The retriever-type axis dominates: BM25 gains +7.7 points of MRR over the dense baseline, because many gold questions reference exact module names or subsection titles that BM25 matches lexically. Hybrid RRF achieves the highest Hit@k by capturing both lexical and semantic matches. Increasing k from 5 to 8 in dense retrieval improves Hit@k by ~2 points but does not recover the MRR gap with BM25, confirming that rank quality matters more than recall breadth. Answer accuracy tracks MRR closely, suggesting generation quality is gated by rank-1 retrieval precision.

## 9. Failure Analysis

From the 10 manually classified samples, 3 errors were observed:

- **Retrieval noise causing false refusals (2 cases).** The gold chunk was present in the top-5 retrieved set but ranked below a cluster of topically similar chunks. The LLM, seeing no clear answer in the highest-scored passages, issued the refusal string. A cross-encoder reranker over top-20 would likely surface the gold chunk to rank 1.
- **Citation hallucination (1 case).** On a code-snippet question, the model cited a chunk ID (sec124) that was not in the retrieved set. The citation filter silently dropped the invented ID, so the final answer was returned without a source. This reveals that the LLM occasionally confabulates IDs when the context contains many similar-looking code blocks.
- **Answer accuracy plateau.** Despite Hit@k reaching 0.693, token-level F1 peaks at 0.364. The gap indicates a generation bottleneck: gemma3:4b occasionally paraphrases rather than quotes numbers or exact subsection names, causing partial F1 matches on numerical and temporal questions. A larger or instruction-tuned model would reduce this gap.
- **Top-k ceiling for dense retrieval.** Raising k from 5 to 8 did not recover any additional grounded questions beyond what BM25 already retrieves, confirming the issue is embedding representation quality rather than recall breadth.

## 10. What I Would Improve Next

- **Cross-encoder reranker** (e.g. ms-marco-MiniLM-L-6-v2) over the top-20 hybrid results to improve rank-1 precision and close the MRR gap. Expected to directly reduce the false-refusal failure mode.
- **Metadata-filtered retrieval:** when the question mentions “Module X” or a specific subsection name, pre-filter the Chroma query on the module metadata field before dense search. This eliminates cross-module false positives.
- **LLM-as-judge evaluation** across the full ~182-question gold set to replace the 10-sample manual classification with a scalable correctness signal.
- **Larger generator** (e.g. gemma3:12b or llama3:8b-instruct) to close the answer-accuracy gap; retrieval is already at 0.693 Hit@5, so generation is the active bottleneck.
- **Chunk-size ablation in full/page load modes** to measure the actual impact of window size on this corpus, since structured mode makes all sections too short for chunk-size variation to matter.
- **Query-expansion or HyDE** (Hypothetical Document Embeddings): generate a hypothetical answer and embed it as the query. Particularly promising for numerical and comparison questions where the query phrasing diverges from the document phrasing.